

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117

Name: Challa Priyadarshini

Reg no.: 192311118

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task

Question 1:

Debugging & Optimization of Symmetric Encryption (DES / 3DES)

A. Identify and fix logical and implementation errors

The inconsistent ciphertext for the **same plaintext and key** usually occurs because of errors in the DES key schedule, permutations, padding, or initialization.

Common errors:

1. Incorrect key scheduling

- DES uses a 64-bit key, with 8 parity bits.
- Apply **PC-1** to obtain 56 bits.
- Split into two 28-bit halves: C and D.
- Perform the correct left rotations for each of the 16 rounds.
- Apply **PC-2** to generate each 48-bit round key.

2. Incorrect permutation

- Initial Permutation (IP) must be applied before the 16 DES rounds.
- Final Permutation (FP) must be applied after the rounds.
- Expansion, S-box substitution, and P-permutation must also follow the correct DES tables.

3. Incorrect padding

- DES operates on **8-byte blocks**.
- Data that is not a multiple of 8 bytes must be padded consistently, such as with PKCS#7.

4. Incorrect IV handling

- CBC/CFB/OFB modes require an IV.
- The same IV must be available for decryption.
- For secure encryption, the IV should normally be unpredictable and should not be reused with the same key.

Correct processing flow:

Plaintext → Padding → Block Splitting → Key Schedule → DES Rounds → Permutations → Ciphertext

For decryption:

Ciphertext → Reverse DES Operations → Remove Padding → Plaintext

B. Optimize the code for datasets ≥ 10 MB

For large datasets, processing every bit individually in Python can be slow.

Optimization methods:

1. Process data in **blocks** instead of individual characters.
2. Use `bytes` and `bytearray` instead of repeatedly converting strings.
3. Avoid recalculating the 16 DES round keys for every block.
4. Generate the key schedule **once** and reuse it.
5. Avoid unnecessary list creation and copying.
6. Process large files in **chunks**, for example 64 KB or 1 MB at a time.
7. Use a well-tested cryptographic library rather than implementing DES manually.
8. For modern applications, use **AES-GCM** or **ChaCha20-Poly1305** instead of DES/3DES.

Example optimized structure:

Python Code:

```
def encrypt_large_file(data, key):

    round_keys = generate_round_keys(key)

    result = bytearray()

    for i in range(0, len(data), 8):

        block = data[i:i+8]
```

```
encrypted = des_encrypt_block(block, round_keys)

result.extend(encrypted)

return bytes(result)
```

The important optimization is that `generate_round_keys()` is called once, rather than once for every block.

C. Should DES/3DES be replaced?

Yes.

DES uses a very small **56-bit effective key**, making it insecure against modern brute-force attacks. 3DES improves the effective security compared with DES but is also outdated and inefficient.

A modern cipher such as AES provides:

- Stronger security
- Better performance
- Hardware acceleration on many modern processors
- Better suitability for large datasets
- Support for authenticated encryption modes such as **AES-GCM**

Therefore:

DES → Not recommended

3DES → Legacy technology

AES-GCM → Recommended for modern applications

Replacing DES/3DES with AES-GCM can therefore improve both security and performance.

Question 2:

Performance Analysis of Block Cipher Modes of Operation

The modes considered are:

- ECB – Electronic Codebook
- CBC – Cipher Block Chaining
- CFB – Cipher Feedback
- OFB – Output Feedback

A. Debug incorrect padding and IV handling

1. Padding problem

AES uses a **16-byte block size**.

For CBC and ECB, plaintext must normally be padded to a multiple of 16 bytes.

Using PKCS#7:

Python :

```
ad_length = 16 - (len(data) % 16)
padded_data = data + bytes([pad_length]) * pad_length
```

During decryption:

Python :

```
pad_length = decrypted[-1]
decrypted = decrypted[:-pad_length]
```

Padding must be applied **before encryption** and removed **after decryption**.

2. IV problem

CBC, CFB and OFB require an IV.

The IV used for decryption must be the same IV used for encryption.

A typical structure is:

Encryption:

Plaintext + Key + IV → Ciphertext

Decryption:

Ciphertext + Key + Same IV → Plaintext

For security, the IV should generally be generated randomly for each encryption operation and stored/transmitted alongside the ciphertext.

ECB does not use an IV, but it should not be selected for secure data encryption because identical plaintext blocks produce identical ciphertext blocks.

B. Refactor to minimize computation and memory usage

The inefficient implementation may repeatedly perform:

Padding → Encryption → Conversion → Copying → Concatenation

for every block.

Instead:

1. Generate the key schedule once.
2. Generate the IV once per message.
3. Process data in chunks.
4. Use `bytes/bytearray`.
5. Avoid repeated string conversions.
6. Avoid repeatedly concatenating immutable `bytes`.
7. Use a streaming interface for large files.
8. Use a standard optimized cryptographic library.

Less efficient:

Python :

```
result = b''
```

```
for block in blocks:
```

```
    result = result + encrypt(block)
```

Better:

Python :

```
result = bytearray()
```

```
for block in blocks:
```

```
    result.extend(encrypt(block))
```

```
ciphertext = bytes(result)
```

This reduces unnecessary memory allocation and copying.

C. Compare modes and select the most efficient mode

ECB

- No IV
- Blocks can be processed independently
- Fast and parallelizable
- Not secure for most real-world data
- Identical plaintext blocks produce identical ciphertext blocks

Conclusion: Do not use for secure communication.

CBC

- Requires IV
- Provides confidentiality
- Encryption is sequential
- Decryption can be parallelized
- Requires padding
- Does not provide authentication by itself

Conclusion: Secure only when combined with appropriate authentication, but not the preferred modern choice.

CFB

- Requires IV
- Can operate on streaming data
- Does not require padding in the usual formulation
- Encryption has sequential dependencies

Conclusion: Useful for some streaming applications but not generally the first choice for new systems.

OFB

- Requires IV
- Converts the block cipher into a stream-like cipher
- No padding required
- Encryption/decryption can be efficient
- Reusing the same IV with the same key is unsafe

Conclusion: Efficient in some cases, but modern authenticated encryption is preferable.

Best choice for real-time secure communication

For a new secure communication system, rather than trying to optimize ECB/CBC/CFB/OFB, use an authenticated-encryption mode such as:

AES-GCM

It provides:

- Confidentiality
- Integrity
- Authentication
- Good performance
- Parallel processing opportunities
- Hardware acceleration on many systems

Final conclusion

For Task 1: Replace DES/3DES with **AES-GCM** because it provides substantially better modern security and performance.

For Task 2: Among ECB/CBC/CFB/OFB, performance depends on the workload, but ECB should not be chosen for secure communication. For a modern real-time system, AES-GCM is generally the better practical choice because it combines encryption and authentication efficiently.

Question 3:

Debugging RSA Cryptosystem Implementation

A Python RSA system may fail for large messages or produce incorrect decrypted plaintext because of problems in prime generation, key generation, message size, or modular exponentiation.

A. Identify and Fix Issues

1. Prime Generation Issues

RSA requires two large prime numbers:

p and q

Common errors:

- Choosing small or non-prime values.

- Using predictable random numbers.
- Not testing whether generated numbers are actually prime.
- Choosing p and q too close together.
- Reusing the same primes.

Fix:

- Generate cryptographically random candidate numbers.
- Use a strong primality test such as Miller–Rabin.
- Generate sufficiently large primes.
- Ensure $p \neq q$.

2. Key Generation Issues

RSA calculates:

$$n = p \times q$$

$$\phi(n) = (p - 1)(q - 1)$$

Choose public exponent:

$$e$$

such that:

$$\gcd(e, \phi(n)) = 1$$

Then calculate the private exponent:

$$d = e^{-1} \bmod \phi(n)$$

The keys are:

Public Key = (e, n)

Private Key = (d, n)

Common errors:

- Choosing an invalid e .
- Calculating d incorrectly.
- Using the wrong value of $\phi(n)$.
- Accidentally swapping public and private keys.

Fix:

Python:

```
from math import gcd
if gcd(e, phi) != 1:
    raise ValueError("Invalid public exponent")
d = pow(e, -1, phi)
```

3. Large Message Problem

Basic RSA cannot directly encrypt an arbitrarily large message.

If the numerical message is m , it must satisfy:

$$0 \leq m < n$$

If:

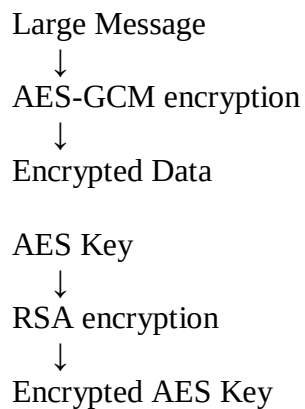
$$m \geq n$$

RSA encryption fails or produces incorrect results.

Solution:

Do not encrypt large files directly using RSA.

Use hybrid encryption:



RSA encrypts only the small symmetric key, while AES encrypts the large message.

4. Modular Exponentiation Issues

RSA encryption is:

$$c = m^e \bmod n$$

Decryption is:

$$m = c^d \bmod n$$

Computing $m^{**} e$ directly is extremely inefficient for large values.

Python's built-in `pow()` is much better:

```
ciphertext = pow(message, e, n)
plaintext = pow(ciphertext, d, n)
```

It performs modular exponentiation efficiently without creating an enormous intermediate number.

B. Optimization Techniques

1. Fast Modular Exponentiation

Instead of:

```
result = (m ** e) % n
```

use:

```
result = pow(m, e, n)
```

This uses efficient modular exponentiation.

The basic idea is **repeated squaring**:

$$m^2 \rightarrow m^4 \rightarrow m^8 \rightarrow m^{16} \rightarrow \dots$$

This reduces the number of multiplication operations significantly.

2. Chinese Remainder Theorem (CRT)

During RSA decryption, CRT can make computation considerably faster.

Calculate:

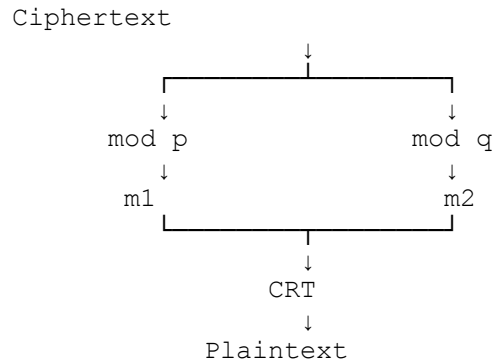
```
dp = d mod (p - 1)
dq = d mod (q - 1)
```

Then:

$$m1 = c^{dp} \bmod p$$
$$m2 = c^{dq} \bmod q$$

The results are combined to obtain the original message.

Conceptually:



CRT works with smaller numbers than directly calculating $c^d \bmod n$, improving RSA decryption performance

3. Optimization Summary

Use:

- Miller–Rabin → efficient prime testing
- $\text{pow}(a, b, n)$ → fast modular exponentiation
- CRT → faster RSA decryption
- Hybrid encryption → handle large messages efficiently
- Efficient byte/block conversion → reduce memory overhead

C. Key Size, Security and Performance

The RSA key size affects both **security and computational cost**.

Small key

1024-bit RSA

- Faster
- Uses less memory
- Lower security
- Not recommended for modern security requirements

Medium key

2048-bit RSA

- Stronger security
- More computationally expensive
- Common minimum choice for many RSA applications

Larger key

3072 / 4096-bit RSA

- Higher security margin
- Slower encryption/decryption
- More CPU and memory usage

Conclusion

There is a trade-off:

Larger RSA Key



Higher Security



Higher Computation



Slower RSA Operations

For modern applications, **RSA-2048 or stronger** may be used depending on the security requirements, while large data should be encrypted using a symmetric cipher such as AES-GCM.

Question 4

Optimization and Security Analysis of Diffie-Hellman Key Exchange

Diffie-Hellman (DH) allows two parties to establish a shared secret over an insecure network.

The basic parameters are:

p = large prime

g = generator

Alice chooses private value:

a

Bob chooses private value:

b

They calculate:

$$A = g^a \bmod p$$

$$B = g^b \bmod p$$

Then:

$$\text{Alice: } K = B^a \bmod p$$

$$\text{Bob: } K = A^b \bmod p$$

Both obtain:

$$K = g^{ab} \bmod p$$

A. Debug Incorrect Shared Key Generation

The most common errors are:

1. Incorrect modular arithmetic

The operation must be:

$$A = \text{pow}(g, a, p)$$

$$B = \text{pow}(g, b, p)$$

Not:

$$A = (g ** a) \% p$$

The pow() version is much more efficient for large values.

2. Wrong shared-key calculation

Alice:

$$\text{shared_alice} = \text{pow}(B, a, p)$$

Bob:

```
shared_bob = pow(A, b, p)
```

The following should be true:

```
assert shared_alice == shared_bob
```

3. Invalid parameters

The implementation should validate:

- p is a suitable large prime.
- g is a valid generator.
- Private values are randomly generated.
- Private values are not reused unnecessarily.

B. Optimize Exponentiation

Large DH values make direct exponentiation extremely expensive.

Inefficient

```
A = (g ** a) % p
```

Efficient

```
A = pow(g, a, p)
```

Python's modular exponentiation calculates the result efficiently without creating the complete huge value $g ** a$.

For repeated operations, also:

- Avoid recalculating the same public values.
- Use optimized cryptographic libraries.
- Use appropriate standardized DH groups.
- Prefer elliptic-curve Diffie-Hellman (ECDH) when suitable.

C. Mitigating Man-in-the-Middle Attacks

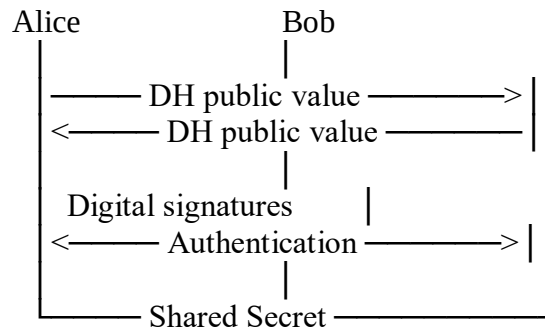
Basic Diffie-Hellman does not authenticate the participants.

An attacker could potentially interfere with the exchange and establish separate keys with Alice and Bob.

Solution: Authentication

Use **authenticated Diffie-Hellman**.

A common approach is:



The DH exchange can be authenticated using a digital-signature system and trusted public keys/certificates.

Another modern approach is to use a standardized authenticated protocol such as **TLS**, which handles key exchange and authentication together.

Performance Impact of Security Enhancements

Adding authentication introduces additional operations:

Basic DH

↓

Modular exponentiation

↓

Shared key

Authenticated DH:

DH

↓

Digital signature generation

↓

Signature verification

↓

Authentication

↓

Shared key

Therefore, authenticated DH requires additional CPU time and communication overhead.

However, this extra cost is justified because basic unauthenticated DH does not protect against active man-in-the-middle attacks.

Final Comparison

Prime Generation



Key Generation



Fast Modular Exponentiation



CRT Optimization



Hybrid Encryption for Large Data

Diffie-Hellman

Choose p and g



Generate private values



Compute public values using $\text{pow}()$



Generate shared secret



Authenticate participants



Use shared key for symmetric encryption

Conclusion

- Use fast modular exponentiation instead of direct exponentiation.
- Use CRT to speed up RSA decryption.
- RSA should not directly encrypt large messages; use hybrid encryption.
- Use sufficiently large RSA keys according to current security requirements.
- Use $\text{pow}(\text{base}, \text{exponent}, \text{modulus})$ for efficient DH calculations.
- Basic DH must be authenticated to prevent man-in-the-middle attacks.
- For modern systems, standardized protocols and well-tested cryptographic libraries are preferable to implementing RSA/DH primitives manually.

Question 5:

Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

ECC is a public-key cryptography technique based on mathematical operations on points over an elliptic curve. It provides strong security with relatively small key sizes.

A. Identify and Fix Errors in Point Addition and Scalar Multiplication

1. Elliptic Curve Equation

A commonly used curve has the form:

where:

- p = prime number
- a and b = curve parameters
- (x, y) = point on the curve

A point must satisfy:

2. Common Point Addition Errors

For two points:

For $P \neq Q$:

Since division is not directly performed modulo p , use the **modular inverse**:

Then:

For Point **Doubling**

When:

the slope becomes:

Then:

Common Bugs

1. Forgetting $\% p$.
2. Performing ordinary division instead of modular inverse.
3. Using the point-addition formula when $P = Q$.
4. Not handling the **point at infinity**.
5. Not handling the case where $x_2 - x_1 \equiv 0 \pmod{p}$.
6. Using curve parameters that do not define a valid curve.

Modular Inverse in Python

```
def mod_inverse(x, p):  
    return pow(x % p, -1, p)
```

This is preferable to calculating a large inverse manually.

3. Point Addition Logic

A simplified implementation is:

Python

```
def point_add(P, Q, a, p):
    if P is None:
        return Q
    if Q is None:
        return P

    x1, y1 = P
    x2, y2 = Q

    # P + (-P) = Point at infinity
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:
        numerator = (3 * x1 * x1 + a) % p
        denominator = (2 * y1) % p
    else:
        numerator = (y2 - y1) % p
        denominator = (x2 - x1) % p

    slope = numerator * pow(denominator, -1, p) % p

    x3 = (slope * slope - x1 - x2) % p
    y3 = (slope * (x1 - x3) - y1) % p

    return (x3, y3)
```

This handles both point addition and point doubling.

B. Optimize Using Double-and-Add

Scalar multiplication means calculating:

k times.

A simple implementation would perform k additions, which is inefficient for a large k.

Double-and-Add Method

Instead, use the binary representation of k .

For example:

$k = 13$

13 in binary = 1101

The algorithm repeatedly:

1. Doubles the current point.
2. Adds the point when the corresponding binary bit is 1.

Optimized Algorithm

Python

```
def scalar_multiply(k, P, a, p):
    result = None
    current = P

    while k > 0:
        if k & 1:
            result = point_add(result, current, a, p)

        current = point_add(current, current, a, p)
        k >>= 1

    return result
```

Why is it faster?

Instead of approximately k additions, double-and-add requires approximately:
point operations.

Therefore:

Simple multiplication:

k additions

Double-and-add:

$\log_2(k)$ doublings + some additions

This gives a major performance improvement for large scalar values.

C. ECC vs RSA

1. Key Size

ECC provides comparable security using much smaller keys.

For example, approximately:

ECC 256-bit $\approx$ RSA 3072-bit

The exact security equivalence depends on the standards and security level being targeted.

2. Computational Efficiency

RSA

RSA uses operations involving very large integers.

Large Key
↓
Large Integer Operations
↓
Higher CPU and Memory Usage

ECC

ECC works with much smaller keys.

Small Key
↓
Elliptic Curve Point Operations
↓
Lower Memory Requirements

With optimized scalar multiplication, ECC can be highly efficient for key exchange and digital signatures.

3. Security Strength

ECC provides strong security with smaller key sizes.

For example:

RSA-2048 → commonly used security level

ECC-256 → comparable modern security level

ECC therefore offers a better **security-to-key-size ratio**.

4. Resource-Constrained Environment

ECC is particularly suitable for:

- IoT devices
- Smart sensors
- Embedded systems
- Mobile devices
- Low-power devices
- Systems with limited memory

Reasons

1. Smaller keys

Less memory is required to store keys and certificates.

2. Lower communication overhead

Smaller public keys require fewer bytes to transmit.

3. Efficient operations

Optimized ECC algorithms such as double-and-add can reduce computational requirements.

4. Lower power consumption

Smaller data sizes and efficient implementations can reduce processing and communication costs.

Conclusion

ECC provides strong security with smaller key sizes and lower memory and computational requirements compared with RSA. By correcting point addition, point doubling, modular

inverse, and point-at-infinity handling, the ECC implementation can produce correct results. Using the double-and-add algorithm significantly improves scalar multiplication efficiency. Therefore, ECC is a better choice than RSA for resource-constrained environments such as IoT and embedded devices, where low memory usage, reduced computation, and efficient communication are important.